International Journal of Computer Engineering and Technology (IJCET)
Volume 11, Issue 6, November-December, 2020, pp. 38-45, Article ID: IJCET_11_06_005
Available online at https://iaeme.com/Home/issue/IJCET?Volume=11&Issue=6
ISSN Print: 0976-6367 and ISSN Online: 0976–6375
Impact Factor (2020): 16.11 (Based on Google Scholar Citation)

© **IAEME** Publication

DESIGNING SALESFORCE LIGHTNING COMPONENTS FOR ENHANCED USER EXPERIENCE

Venkat Sumanth Guduru

ABSTRACT

Lightning Web Components (LWC) is the new arresting development in the Salesforce UI and UX framework; this paper discusses it. LWC was introduced by salesforce in 2019 and it is the new way of creating web applications without relying on old web methodologies and frameworks and with integration to the latest one – JavaScript. This paper also discusses some of the features of LWC: performance, adaptation to modern JavaScript standards, and progress towards a less complex framework.

Component-based architecture in LWC not only improvements performance and scalability but also helps in modularity, reusability. Such attributes are convenient for the developers to handle and modify applications while the interfaces remain coherent, logical, and unify in design. This paper also provides an overview of how to create LWC best practices based on their structure for the components, reusability, modularity, and accessibility. In this way developers are not going to stick in the trap and instead are going to build the components that are strong, maintainable and easy to use.

This paper is this guide to implementing LWC in salesforce applications Depending on the section, this paper uses, pseudocode, flowcharts, and architecture diagrams to explain an implementation of LWC. As we see that digital landscape changes, LWC present itself as an essential tool for developers and organizations that strive to provide solutions for today's world. In other words, LWC presents the new state-of-art CRM solutions best practices and reaffirms Salesforce leading position on the market.

Cite this Article: Venkat Sumanth Guduru, Designing Salesforce Lightning Components for Enhanced User Experience, International Journal of Computer Engineering and Technology (IJCET), 11(5), 2020, pp. 38-45.
https://iaeme.com/Home/issue/IJCET?Volume=11&Issue=6

Keywords

Lightning Web Components (LWC): UI framework designed and implemented by the salesforce for creating UI using modern web standards and JavaScript. This is possible because it provides a faster development and enhancement of the Salesforce apps.

Salesforce: A CRM sales tool that operates from the cloud and provides features for sales, services for marketing and others. It offers sets of features and services to assist with the communication of businesses with their clients and handling of data.

User Interface (UI): The place where a user directly comes in contact with the digital device or using the software application. It entails all the graphical representations, widgets, and components that a user employs in an application.

User Experience (UX): Can be defined as the general perception and experience of a user in relation to a specific product or a system. It includes all that the end-user is likely to encounter; from the usability, the designing and even the functionality.

Component-based Architecture: The practice of constructing software systems that are based on the reusability of the components. All of them fold a particular purpose and can be linked together to form more elaborate programs.

Modular Design: A system design process that breaks down a complex system into smaller constituent units of deployment referred to as modules. As each of the modules is separated, it becomes quite easy to add, modify, or update one feature without affecting the others.

Accessibility: The process of ensuring that the content and applications developed for digital ecosystem are accessible for disabled persons. It entails the creation of the websites and applications which can be employed by persons who have disabilities as well as those of ordinary ability.

Cloud Computing: Computing services that can be accessed through the internet which may include servers, storage, database, networking, software, and analytics. It delivers versatile assets which can be applied in many areas of technology; faster creation of new products; and numerous scale efficiencies.

I. INTRODUCTION

Salesforce is a renowned global player in the provision of customer relationship management (CRM) solutions, particularly for having one of the most fluid capabilities in adapting to changing UI/UX trends. Among the improvements in this evolutionary model, one stands out and that is the incorporation of LWC which is also known as Lightning Web Components. LWC was introduced in 2019 as a radical improvement to a solution for constructing Saleforce applications that are faster, more effective, and more backward-compatible. The existing web standards and the enhancements in JavaScript make LWC lightweight, easier to develop as well as faster and more capable.

As for the main idea of this paper, it considers how LWC is changing the UI/UX framework in Salesforce. Here, we examine the essential characteristics and advantages of LWC compared to prior Lightning Web Components and discuss the solution's influence on the Figure 2: LWC pre- and post-release user interface usability and developers' efficiency. In this paper, the focus will be made on the consideration of the key aspects of the design of the Lightning components with the description of the primary features, advantages, typical mistakes, and the given case-study.

This also involves analysis of the benefits of using LWC in order to improve interactivity, customization together with the performance to make LWC as a cornerstone tool of choice for developers and business that aim at the greatest heights in the Salesforce environment.

II. FUNDAMENTALS OF LIGHTNING WEB COMPONENTS (LWC)

What is LWC?

LWC is one of the development models that help in developing Salesforce applications and the modern JavaScript frame work. LWC takes benefit of web standards and advanced JavaScript along with the component architecture and reusability factor. This is different from Aura Components that was used in Salesforce and which is relatively heavier than this one with more complex way of learning and lesser performance tips.

Key Features of LWC

Performance improvements. Purposing the significance of LWC, it is possible to state that it defines a modern framework to develop the Salesforce applications that are more effective, fast and easy to use. Upgrading from an LWC to another version of LWC is arguably one of the main benefits since it entails performance enhancements[1]. LWC works seamlessly with native browser features to reduce the amount of work that takes place during component instantiation, rendering and binding with the data which is a factor that enhances the optimization of load time as well as the general responsiveness of the interface.

Modern JavaScript support. Another important aspect here is support for more recent versions of JavaScript. LWC is developed completely with the modern ECMAScript Standards, ES6 modules, classes, async/await functions. This helps developers to write less complicated code and easy to maintain, because of the advance features and addition functionality.

Enhanced developer experience. LWC also has improved developer experience. New, lighter, and more flexible due to its modularity, the architectural organization of this new model allows for better development. As mentioned, the software developers are capable of using a coherent and to some extent simple strategy which makes it possible to implement seamlessly into many development tools and frameworks. This makes the development's work structure more organized and also promotes sharing of components between developers without much strain.

III. REVOLUTIONIZING UI/UX WITH LWC

Performance Enhancements

The most exciting aspect that LWC offers in the area of UI/UX is the aspect of performance. Taking advantage of the browser capabilities and the latest JavaScript, LWC components are faster and more optimized. This consists of LWC features such as Shadow DOM where styles and scripts are confined in the component thereby minimizing on the side effects and enhances the rendering. This encapsulation makes the component load faster to minimize the time spent on a re-rendering process hence a more enhanced user experience.

[1.] [1] Yin, J. (2019). Salesforce-Usability of Lightning Web Components

User Experience Improvements

Component-based architecture of LWC has benefits on the user experience since it makes the application modular and reusable. In LWC components, everything is built to be independent of one another with the possibility to be reused; this means that while building a sophisticated user interface, everything is built as an assemblage of small parts. This modularity not only contributes to the easier creation of the different components, but also facilitates the making of the components to be similar throughout the numerous sections of the application.

Additionally, LWC can include such special elements as custom, as well as declarative rendering which improve interactivity, as well as possibilities of customization. It is now possible to make an easy and fast response to any component, so the usage of components becomes much more interesting.

Developer Experience

In particular, LWC is easy to develop compared to its predecessors; for developers, the creation of LWC is easier and faster. Promoting the use of more contemporary features of the JavaScript results in the production of less code in components and makes them more comprehensible and less complicated to maintain. LWC also fits with IDEs such as Salesforce CLI, Visual Studio, and other testing means and libraries, which make the development process more effective and the code more efficient.

IV. DESIGNING EFFECTIVE LIGHTNING COMPONENTS

Best Practices for LWC Design

LWC is one of the important aspects of the Salesforce application, and here in this article, you will learn effectively, how to design lightning web components. Here are three crucial best practices

Component Structure and Organization: In structuring of the LWC components there are certain rules which must be strictly followed and they include the following. Everty module should be created for a given purpose and thus serves a single purpose and therefore can be debugged easily. Good component organization makes it easier to distinguish functionality and helps in avoiding cluttering of a code[2]. Components should use standard naming and directory conventions and should be easy to read and understand and developed and modified jointly.

Reusability and Modularity: LWCs should be made re-usable so that when a developer is writing code in a place and also wants to use the same component somewhere else he or she does not have to write the whole code again. This means coming up with modules which can receive properties in order to accept dynamic data and also send out events in order to inform of changes. Modular components are independent of each other and as such do not depend heavily on each other; this makes it easy to change or update one component without affecting others. Stress on modularity also makes it easy to test the software and also there is little chance to bring new bugs into system when modifying.

[2.] [2] https://developer.salesforce.com/blogs/2020/08/error-handling-best-practices-for-lightning-web-components

Accessibility Considerations: Making sure that the components are equally accessibly for all users irrespective of the disability is one of the pre-requisites to be met while designing LWC. An accessibility focus can be established by using semantic HTML elements and following guidelines as the Web Content Accessibility Guidelines[5]. Three important things are worn to make the components of an application accessible include giving the components descriptive labels, ensuring that they are keyboard accessible, and applying ARIA roles and properties where required. Speaking of accessibility, it need to be pointed out that, in addition to explicitly helping persons with disabilities, it also positively affects the usability and perceived satisfaction with the interface.

V. COMMON PITFALLS AND HOW TO AVOID

Performance Bottlenecks: This is however not to say that LWC developers do not experience performance problems; they do so where the components are not well implemented. Some of the relatable vices include overreliance of the application on reactive properties; the ineffective data management; or the improper application of the Shadow DOM[3]. In order to avoid such problems, one has to adhere to the rules of performance degeneration, reactivity minimization, optimization of database requests and proper usage of Shadow DOM.

Usability Issues: It is possible to experience such problems as usability if the components were not developed oriented on the end user. This entails issues to do with its aesthetic, its interaction and general lack of adaptability such as the following: To these challenges, developers should attend to user interface testing, to follow recommended guidelines, and to make sure that those parts offer simplistic and harmonious usability[4].

[3.] 3 Flanagan, D. (2020). *JavaScript: The Definitive Guide: Master the World's Most-Used Programming Language*. O'Reilly Media.

[4.]4https://developer.salesforce.com/blogs/2020/11/ecosystem-innovation-with-lightning-web-components

[5.]5https://developer.salesforce.com/blogs/2020/12/customizing-your-lightning-experience-with-styling-hooks

VI. LWC IMPLEMENTATION

Pseudo code for a Sample LWC Component

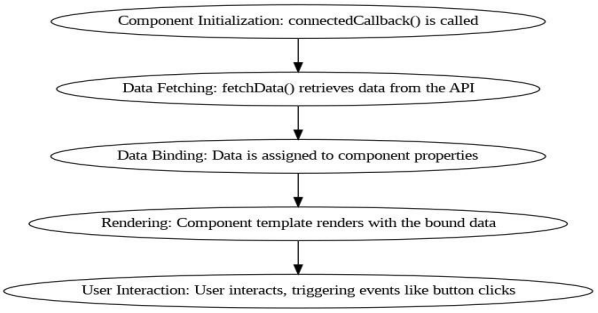
```
// LWC Component: accountList

// HTML file (accountList.html)
<template>
  <!-- Display a spinner while data is loading -->
  <lightning-spinner if:true={isLoading}></lightning-spinner>

  <!-- If accounts are available, display the account list -->
  <template if:true={accounts}>
    <ul>
      <!-- Loop through the account list and display each account name -->
      <template for:each={accounts} for:item="account">
        <li key={account.Id}>{account.Name}</li>
      </template>
    </ul>
  </template>

  <!-- Display a message if no accounts are found -->
  <template if:false={accounts}>
    <p>No accounts found.</p>
  </template>
</template>
```

```
// JavaScript file (accountList.js)
import { LightningElement, wire } from 'lwc';
import getAccounts from '@salesforce/apex/AccountController.getAccounts';

export default class AccountList extends LightningElement {
  // Boolean property to track loading state
  isLoading = true;

  // Property to store account data fetched from Apex
  accounts;

  // Fetch accounts using the wire service, call Apex method
  @wire(getAccounts)
  wiredAccounts({ error, data }) {
    // If data is fetched successfully, set the account data and stop loading
    if (data) {
      this.accounts = data;
      this.isLoading = false;
    } else if (error) {
      // Handle errors
      console.error('Error fetching accounts:', error);
      this.isLoading = false;
    }
  }
}

// Apex controller (AccountController.cls)
public with sharing class AccountController {
  @AuraEnabled(cacheable=true)
  public static List<Account> getAccounts() {
    // Query the list of accounts
    return [SELECT Id, Name FROM Account LIMIT 10];
  }
}
```

Here is a basic example of an LWC component that displays a list of accounts from salesforce:

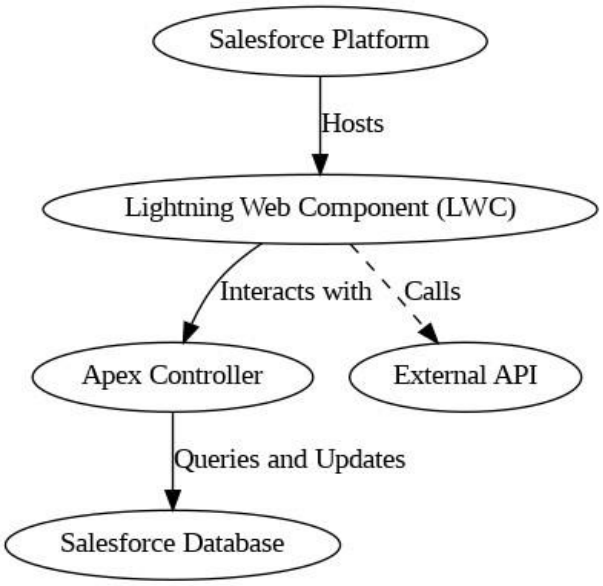
Component Workflow

The flowchart below illustrates the lifecycle of the LWC component, from initialization to user interaction and data rendering:



Architecture Diagram

The architecture diagram below shows how the LWC component integrates with Salesforce backend services:



VII. CONCLUSION

Salesforce LWC has particularly helped the Salesforce development spectrum with better, optimized, and customer-focused approach. LWC is based upon the native browser features as well as the most modern in JavaScript libraries; this way, LWC is characterized by the utmost speed, scalability, as well as the interactivity of UI that the developers can leverage to the maximum. LWC follows the component development approach which makes it more reliable, easier to maintain and more fluid over UI and UX over all the platforms[7].

In addition, as a result of emphasis on its availability and incorporation of the standards, applications for which LWC is of great value are also available to populations of varied abilities. Considering the ongoing modifications of the Salesforce platform LWC is one of those tools that will define the future of the UI/UX in Salesforce[6].

If an organization or a developer decides to go with LWC then they will be at par with other market requirements as LWC inculcates solutions with in the process. Summing up, LWCs are not only the technological development, but the change of the paradigm of concepts and methods of creating new ideas and strategies in organizing the usage of the Salesforce applications and setting new standards in the field of CRM.

REFERENCES

[1] Yin, J. (2019). Salesforce-Usability of Lightning Web Components.

[2] https://developer.salesforce.com/blogs/2020/08/error-handling-best-practices-for-lightning-web-components.

[3] Flanagan, D. (2020). JavaScript: The Definitive Guide: Master the World's Most-Used Programming Language. O'Reilly Media.

[4] https://developer.salesforce.com/blogs/2020/11/ecosystem-innovation-with-lightning-web-components.

[5] https://developer.salesforce.com/blogs/2020/12/customizing-your-lightning-experience-with-styling-hooks.

[6] Burba, M. A. (2020). Organization of communication between Salesforce customers and clients.

[7] Davis, A., & Davis, A. (2019). Developing on Salesforce. Mastering Salesforce DevOps: A Practical Guide to Building Trust While Delivering Innovation, 67-108.